

CS6008: Network Security — Assignment 1

Gunda Sushanth (23b1063)
Boyina Sanjana (23b0976)

1 System Architecture & Deployment Topology

The chat system was deployed across four separate virtual machines (VMs) running Ubuntu Linux over a shared bridged subnet. Operating in distinct virtual machines guarantees authentic network latency, packet serialization, and prevents loopback-based side channels.

Node Role	Host Identity / IP	Port	Cryptographic Function
Chat Server	192.168.1.126	8080	Central relay, link-DH peer & PKI target
Client 1 (Alice)	192.168.1.124	Dynamic	Initiating client node & E2E participant
Client 2 (Bob)	192.168.1.123	Dynamic	Peer client node & E2E participant
Attacker (Mallory)	192.168.1.125	8080	Active MITM proxy & attack tool

Table 1: Deployment Network Topology across 4 Isolated Virtual Machines.

2 Phase 1: Baseline Plaintext Chat System

2.1 Protocol Design & Message Framing

The baseline application implements a centralized client-server model. To prevent TCP stream coalescing and message fragmentation issues inherent in stream-oriented transport protocols, our protocol employs delimiter-based framing using newline characters (`\n`). The socket reader dynamically buffers partial TCP chunks and extracts complete messages only when a delimiter is encountered.

- **Client Registration:** Upon TCP connection establishment, the client transmits `REGISTER <username>\n`. The server validates uniqueness and registers the socket descriptor in a thread-safe registry.
- **Direct Routing:** Messages are formatted as `@<recipient> <payload>\n`. The server parses the target username, looks up the destination socket, and relays `FROM @<sender>: <payload>\n`.
- **User Management:** Commands include `/who` (listing active online users), `/chat <user>` (setting default recipient context), and `/quit` (graceful teardown).

2.2 Plaintext Vulnerability Verification

As mandated by the problem specification, the server was instrumented to log all relayed payloads. Furthermore, Wireshark packet capture was executed on the network interface (`enp0s1`).

```

kittu@kittu: ~/Desktop/Chat_Application/phase1
kittu@kittu: ~/Desktop/Chat_Application/phase1$ ./server 8080
=====
CS6008 Phase 1 Plaintext Chat Server
Listening on port: 8080
=====
[INFO] New connection from 192.168.1.124:50054 (socket 4)
[REGISTRATION] Client on socket 4 registered as 'alice'
[INFO] New connection from 192.168.1.123:32868 (socket 5)
[REGISTRATION] Client on socket 5 registered as 'bob'
[RELAY] From [alice] to [bob]: hi
[RELAY] From [alice] to [bob]: hello
[RELAY] From [bob] to [alice]: hi
[RELAY] From [bob] to [alice]: hello

```

Figure 1: Phase 1: Server terminal logging plaintext chat relays between Alice and Bob.

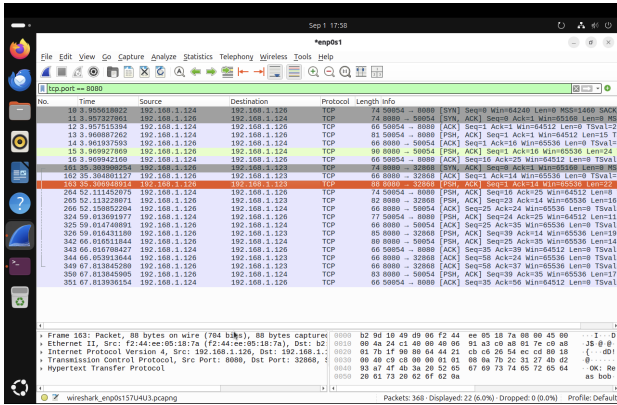


Figure 2: Phase 1: Wireshark capture filtered on tcp.port == 8080.

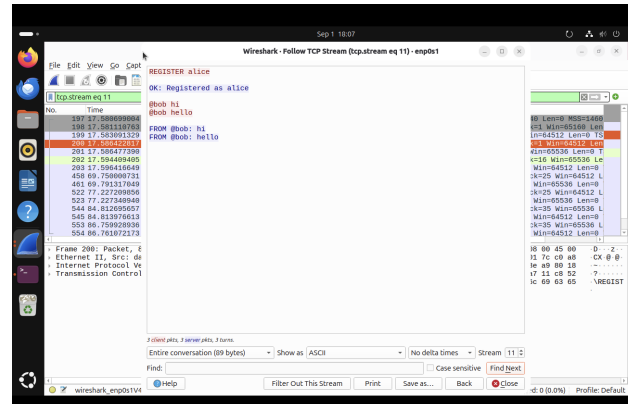


Figure 3: Phase 1: Wireshark Follow TCP Stream showing unencrypted text.

As shown in Figures 1, 2, and 3, passive eavesdroppers and intermediate network entities have complete, uninhibited visibility into private communications and registration identifiers.

3 Phase 2: Link Confidentiality & MITM Attack

3.1 From-Scratch Diffie-Hellman Implementation

In accordance with course rules prohibiting high-level wrapper APIs (`<openssl/ssl.h>` and `<openssl/dh.h>`), Diffie-Hellman key agreement was built from scratch using low-level BigNumber arithmetic (`<openssl/bn.h>`) on the standardized **RFC 3526 Group 14** 2048-bit MODP prime (p) with generator $g = 2$.

1. Each party generates a cryptographically secure 2048-bit private exponent $a \in [2, p - 2]$ via `BN_rand()`.
2. Public values $A = g^a \bmod p$ and $B = g^b \bmod p$ are computed via `BN_mod_exp()` and exchanged as hex strings.
3. Both parties compute the identical raw shared secret $s = B^a \bmod p = A^b \bmod p$.

3.2 Cryptographic Key Derivation Function (KDF)

Theoretical Rationale: The raw DH shared secret $s \in \mathbb{Z}_p^*$ cannot be used directly as an AES key. The distribution of elements produced by modular exponentiation in finite cyclic subgroups does not exhibit uniform bit entropy; certain bit positions have mathematical biases and algebraic dependencies.

To transform the non-uniform secret into a cryptographically strong symmetric key, we apply the SHA-256 hash function as a randomness extractor:

$$K_{\text{AES}} = \text{SHA-256}(s_{\text{raw}}) \in \{0, 1\}^{256} \quad (1)$$

The printable session key fingerprint is defined as $\text{SHA-256}(K_{\text{AES}})$.

3.3 Authenticated Encryption: AES-256-GCM

Unauthenticated stream or block ciphers (e.g., AES-CBC, AES-CTR) are vulnerable to bit-flipping and ciphertext tampering. We utilize AES-256 in Galois/Counter Mode (GCM) via `<openssl/evp.h>`, providing both confidentiality and data integrity. Each packet incorporates:

1. A fresh 12-byte random Initialization Vector (IV) generated via `RAND_bytes()`.
2. Ciphertext produced via CTR mode encryption.
3. A 16-byte Galois Message Authentication Code (GMAC) tag over $GF(2^{128})$.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./client 192.168.1.126 8080 alice
[INFO] Connected to 192.168.1.126:8080
[CRYPTO] Initiating Diffie-Hellman Key Exchange...
[CRYPTO] Diffie-Hellman Key Exchange Complete!
Session Key Fingerprint: 9219a835745da328b58ddc65f969e6171b2fe0fd7c984d
7f78ef26c1b19d1379
=====
Commands:
@username <msg> - Send encrypted message to user
/chat <username> - Switch active chat partner
/who - List online users
/tamper <msg> - Send tampered ciphertext (Tamper Test)
/quit - Disconnect and exit
=====
>
OK: Registered as alice
> @bob Hi
> hello
FROM @bob: Hi
>
FROM @bob: hello
>

```

Figure 4: Phase 2: Alice client session key fingerprint.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./client 192.168.1.126 8080 bob
[INFO] Connected to 192.168.1.126:8080
[CRYPTO] Initiating Diffie-Hellman Key Exchange...
[CRYPTO] Diffie-Hellman Key Exchange Complete!
Session Key Fingerprint: fcf4f4277a023824cdabe8ccb1e551506cbf093dfb9171
2b16f6fbafcbcd6b78
=====
Commands:
@username <msg> - Send encrypted message to user
/chat <username> - Switch active chat partner
/who - List online users
/tamper <msg> - Send tampered ciphertext (Tamper Test)
/quit - Disconnect and exit
=====
>
OK: Registered as bob
>
FROM @alice: Hi
>
FROM @alice: hello
> @alice Hi
> hello
>

```

Figure 5: Phase 2: Bob client session key fingerprint.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./server 8080

=====
CS6008 Phase 2 Secure Chat Server (DH+AES-GCM)
Listening on port: 8080
=====

[INFO] New connection from 192.168.1.124:52612 (socket 4)
[CRYPTO] DH exchange successful with socket 4
      Session Key Fingerprint: 9219a835745da328b58ddc65f969e6171b2fe0fd7c984d
7f78ef26c1b19d1379
[REGISTRATION] Client on socket 4 registered as 'alice'

[INFO] New connection from 192.168.1.123:39636 (socket 5)
[CRYPTO] DH exchange successful with socket 5
      Session Key Fingerprint: fcf4f4277a023824cdabe8ccb1e551506cbf093dfb9171
2b16f6bfabcbed6b78
[REGISTRATION] Client on socket 5 registered as 'bob'

[RELAY] From [alice] to [bob]: Hi
[RELAY] From [alice] to [bob]: hello
[RELAY] From [bob] to [alice]: Hi
[RELAY] From [bob] to [alice]: hello

```

Figure 6: Phase 2: Server matching session fingerprints.

3.4 Integrity Verification: Tamper Test

A dedicated diagnostic command (`/tamper <msg>`) was implemented to flip the final byte of the ciphertext prior to wire transmission. Upon receipt, the server calls `EVP_DecryptFinal_ex()`, which validates the GMAC tag.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./server 8080

=====
CS6008 Phase 2 Secure Chat Server (DH+AES-GCM)
Listening on port: 8080
=====

[INFO] New connection from 192.168.1.124:52612 (socket 4)
[CRYPTO] DH exchange successful with socket 4
      Session Key Fingerprint: 9219a835745da328b58ddc65f969e6171b2fe0fd7c984d
7f78ef26c1b19d1379
[REGISTRATION] Client on socket 4 registered as 'alice'

[INFO] New connection from 192.168.1.123:39636 (socket 5)
[CRYPTO] DH exchange successful with socket 5
      Session Key Fingerprint: fcf4f4277a023824cdabe8ccb1e551506cbf093dfb9171
2b16f6bfabcbed6b78
[REGISTRATION] Client on socket 5 registered as 'bob'

[RELAY] From [alice] to [bob]: Hi
[RELAY] From [alice] to [bob]: hello
[RELAY] From [bob] to [alice]: Hi
[RELAY] From [bob] to [alice]: hello

Commands:
@username <msg> - Send encrypted message to user
/chat <username> - Switch active chat partner
/who - List online users
/tamper <msg> - Send tampered ciphertext (Tamper Test)
/quit - Disconnect and exit

>
OK: Registered as alice
> @bob Hi
> hello
FROM @bob: Hi
>
FROM @bob: hello
> /tamper this is a tampered msg
[TEST] Sending intentionally tampered ciphertext to test AES-GCM integrity...
[SECURITY TEST] Tampered with ciphertext byte before sending!
>
[INFO] Connection lost or session terminated.

```

Figure 8: Phase 2: Alice executing `/tamper` command.

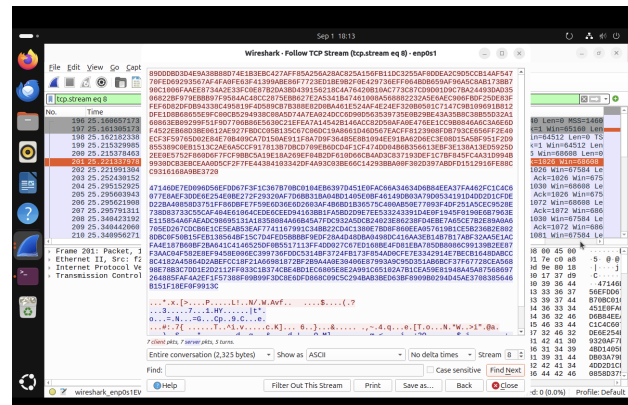


Figure 7: Phase 2: Wireshark encrypted payload capture.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./server 8080

=====
CS6008 Phase 2 Secure Chat Server (DH+AES-GCM)
Listening on port: 8080
=====

[INFO] New connection from 192.168.1.124:52612 (socket 4)
[CRYPTO] DH exchange successful with socket 4
      Session Key Fingerprint: 9219a835745da328b58ddc65f969e6171b2fe0fd7c984d
7f78ef26c1b19d1379
[REGISTRATION] Client on socket 4 registered as 'alice'

[INFO] New connection from 192.168.1.123:39636 (socket 5)
[CRYPTO] DH exchange successful with socket 5
      Session Key Fingerprint: fcf4f4277a023824cdabe8ccb1e551506cbf093dfb9171
2b16f6bfabcbed6b78
[REGISTRATION] Client on socket 5 registered as 'bob'

[RELAY] From [alice] to [bob]: Hi
[RELAY] From [alice] to [bob]: hello
[RELAY] From [bob] to [alice]: Hi
[RELAY] From [bob] to [alice]: hello

[ALERT] AES-GCM Integrity Check Failed! Packet tampered with or corrupted.
[DISCONNECT] User 'alice' disconnected or session terminated.

```

Figure 9: Phase 2: Server rejecting tampered ciphertext.

As shown in Figure 9, the server immediately flags `[ALERT] AES-GCM Integrity Check Failed!` and terminates the compromised socket, preventing forged or manipulated payloads.

3.5 The Man-in-the-Middle (MITM) Attack

Because unauthenticated Diffie–Hellman exchanges public keys in the clear without cryptographic binding to node identities, an active attacker positioned on the network path can intercept and split the key exchange.

We implemented an active MITM proxy tool (`mallory_proxy`). When Alice connects to Mallory (believing Mallory is the server), Mallory establishes two distinct, independent DH handshakes:

1. Handshake 1 ($C1 \leftrightarrow M$): Negotiates Key K_1 .

2. Handshake 2 ($M \leftrightarrow S$): Negotiates Key K_2 .

When Alice sends a message encrypted under K_1 , Mallory decrypts it to plaintext, logs it, re-encrypts under K_2 , and forwards it to the Server.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./server 8080
=====
CS6008 Phase 2 Secure Chat Server (DH+AES-GCM)
Listening on port: 8080
=====

[INFO] New connection from 192.168.1.125:42912 (socket 4)
[CRYPTO] DH exchange successful with socket 4
      Session Key Fingerprint: 7ecf1c2d6f4f8e6da1ccd7e4a9aff3fd0f45a88ab1c7c
c36979ddd841120b97
[REGISTRATION] Client on socket 4 registered as 'alice'

[INFO] New connection from 192.168.1.123:33720 (socket 5)
[CRYPTO] DH exchange successful with socket 5
      Session Key Fingerprint: 002e39ee818be78b44d3830e60e06a87d64f007ce63815
e79f6a67e0bcea2ae7
[REGISTRATION] Client on socket 5 registered as 'bob'
[RELAY] From [alice] to [bob]: super secret intercepted msg

```

Figure 10: Phase 2: Server communicating with Mallory.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./client 192.168.1.125 8080 alice
[INFO] Connected to 192.168.1.125:8080
[CRYPTO] Initiating Diffie-Hellman Key Exchange...
[CRYPTO] Diffie-Hellman Key Exchange Complete!
      Session Key Fingerprint: 8ab17ada2740e876d8edfcb9867dcaff0f25e489f8fafa
6da407b4e7563deb54
=====
Commands:
@username <msg> - Send encrypted message to user
/chat <username> - Switch active chat partner
/who - List online users
/tamper <msg> - Send tampered ciphertext (Tamper Test)
/quit - Disconnect and exit
=====
>
OK: Registered as alice
> @bob super secret intercepted msg
>

```

Figure 11: Phase 2: Alice connected through Mallory proxy.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./client 192.168.1.126 8080 bob
[INFO] Connected to 192.168.1.126:8080
[CRYPTO] Initiating Diffie-Hellman Key Exchange...
[CRYPTO] Diffie-Hellman Key Exchange Complete!
      Session Key Fingerprint: 002e39ee818be78b44d3830e60e06a87d64f007ce63815
e79f6a67e0bcea2ae7
=====
Commands:
@username <msg> - Send encrypted message to user
/chat <username> - Switch active chat partner
/who - List online users
/tamper <msg> - Send tampered ciphertext (Tamper Test)
/quit - Disconnect and exit
=====
>
OK: Registered as bob
>
FROM @alice: super secret intercepted msg
>

```

Figure 12: Phase 2: Bob connected to Server.

```

kittu@kittu: ~/Desktop/Chat_Application/phase2
kittu@kittu: ~/Desktop/Chat_Application/phase2$ ./mallory_proxy 8080 192.168.1.12
6 8080
=====
CS6008 Phase 2 MITM Attack Proxy (Mallory)
Listening on port 8080 -> Forwarding to 192.168.1.126:8080
=====

[MALLORY] Victim client connected! Starting MITM attack...
[MALLORY] Key 1 established with Client! Fingerprint: 8ab17ada2740e876d8edfcb986
7dcaff0f25e489f8fafa6da407b4e7563deb54
[MALLORY] Key 2 established with Server! Fingerprint: 7ecf1c2d6f4f8e6da1ccd7e4a9
aff3fd0f45a88ab1c7cc36979ddd841120b97
[MALLORY] Proxy active. Eavesdropping on all decrypted messages below:

[MALLORY INTERCEPT] (Client -> Server): REGISTER alice
[MALLORY INTERCEPT] (Server -> Client): OK: Registered as alice
[MALLORY INTERCEPT] (Client -> Server): @bob super secret intercepted msg

```

Figure 13: Phase 2: Mallory intercepting cleartext chat.

Observable Evidence of Compromise: An ordinary user typing chat messages has no visual cue of the attack since messages are delivered seamlessly. However, the attack is detectable by comparing session key fingerprints:

- Alice Key Fingerprint: 8ab17ada2740... (matches Mallory K_1).
- Server Key Fingerprint: 7ecf1c2d6f4f... (matches Mallory K_2).

The disparity between Alice's fingerprint and the Server's fingerprint is definitive mathematical proof of an active MITM interception.

4 Phase 3: PKI Authentication & Proof-of-Possession

4.1 Public Key Infrastructure (PKI) Setup

To defeat the MITM vulnerability exposed in Phase 2, we established a Public Key Infrastructure using OpenSSL:

1. **Root CA:** A self-signed Root Certificate Authority (`ca.crt`) with 4096-bit RSA key (`ca.key`) was generated.
2. **Server Certificate:** The chat server generated a 2048-bit RSA private key (`server.key`) and a Certificate Signing Request (CSR) with Common Name `CN = chat.server.local`. The CSR was signed by the Root CA to produce `server.crt`.

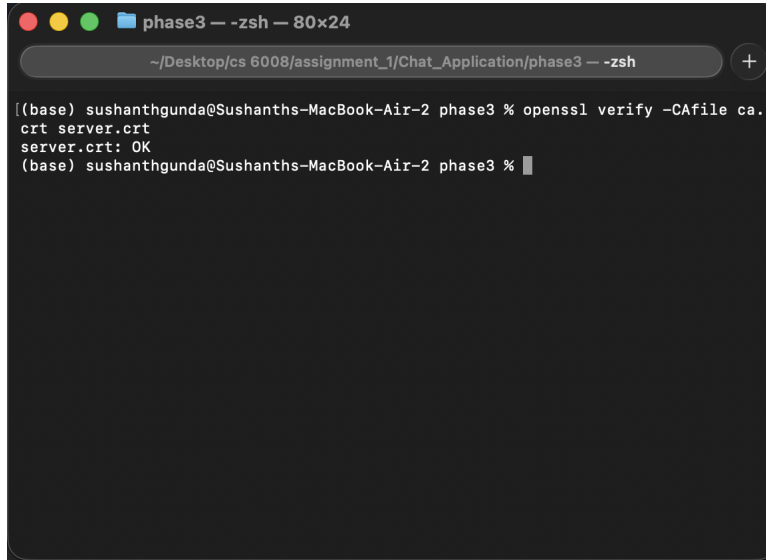


Figure 14: Phase 3: OpenSSL CLI validation of Server certificate against Root CA.

4.2 Client Certificate Validation Chain

During connection initiation, the server sends its PEM-encoded `server.crt`. The client verifies the certificate using `<openssl/x509.h>` by checking:

1. **Signature Validity:** Verified against the trusted local `ca.crt` via `X509_verify_cert()`.
2. **Temporal Validity:** Checks `notBefore` and `notAfter`.
3. **Identity Binding:** Confirms `Subject CN == chat.server.local`.

4.3 Vouched Identity vs. Proof-of-Possession (PoP)

Theoretical Rationale: Certificate validation alone only proves that `server.crt` is a valid document issued by the trusted CA. Because public certificates are openly distributable, an attacker could steal `server.crt` and replay it to clients.

To prove that the remote party actually owns the matching private key, we implement a dynamic Proof-of-Possession challenge-response handshake:

1. Client generates a cryptographically random 32-byte nonce N_{client} and sends it to the server.
2. Server signs the concatenated challenge ($N_{\text{client}} \parallel \text{DH_Pub}_{\text{Server}}$) using its RSA private key via `EVP_DigestSign (RSA-SHA256)`.
3. Client extracts the public key from `server.crt` and verifies the signature using `EVP_DigestVerify`.

Signing the server's ephemeral DH public key inside the challenge firmly binds the key exchange to the authenticated server identity.


```
kittu@kittu: ~/Desktop/Chat_Application/phase3
kittu@kittu: ~/Desktop/Chat_Application/phase3$ ./client 192.168.1.126 8080 alice
ca.crt chat.server.local
[INFO] Connected to 192.168.1.126:8080
[PKI] Initiating Certificate Exchange & Proof-of-Possession validation...
[PKI VALIDATION PASSED] Server certificate signed by trusted Root CA for CN='chat.server.local'
[PoP VERIFICATION PASSED] Server proved possession of private key matching certificate!
[CRYPTO] DH Session Key Established! Fingerprint: 0281abb24d1dde66da931902cc2ad9
022a50a69d3a7c2e8bef77846da225a665
=====
Commands:
@username <msg> - Send encrypted message to user
/chat <username> - Switch active chat partner
/who - List online users
/quit - Disconnect and exit
=====
>
OK: Registered as alice
>
```

Figure 15: Phase 3: Client validating PKI & PoP.

```
kittu@kittu: ~/Desktop/Chat_Application/phase3
kittu@kittu: ~/Desktop/Chat_Application/phase3$ ./server 8080 server.crt server.key
=====
CS6008 Phase 3 PKI-Authenticated Chat Server
Certificate: server.crt
Listening on port: 8080
=====
[INFO] New connection from 192.168.1.124:37684 (socket 4)
[PKI & CRYPTO] Server authenticated successfully by client!
Derived Session Key Fingerprint: cd89c7499c4a7a3ae9414274127b3366
a46d7d3568c95f45ad49840f783a5585
[REGISTRATION] Client on socket 4 registered as 'alice'
```

Figure 16: Phase 3: Server confirming authentication.

4.4 Defeating MITM Attacks

We evaluated two distinct attack vectors using our updated `mallory_proxy`:

4.4.1 Test 1: Mallory Self-Signed Fake Certificate

Mallory attempts to present a forged certificate signed by an untrusted CA. As shown in Figures 17 and 18, the client detects the untrusted certificate chain, halts execution with `[PKI VALIDATION FAILED]`, and closes the socket before exposing any DH parameters.

```
kittu@kittu: ~/Desktop/Chat_Application/phase3
kittu@kittu: ~/Desktop/Chat_Application/phase3$ ./client 192.168.1.125 8080 alice
ca.crt chat.server.local
[INFO] Connected to 192.168.1.125:8080
[PKI] Initiating Certificate Exchange & Proof-of-Possession validation...
[PKI VALIDATION FAILED] Failed to parse PEM X.509 certificate.
[ABORT] Terminating connection immediately without proceeding to DH!
kittu@kittu: ~/Desktop/Chat_Application/phase3$
```

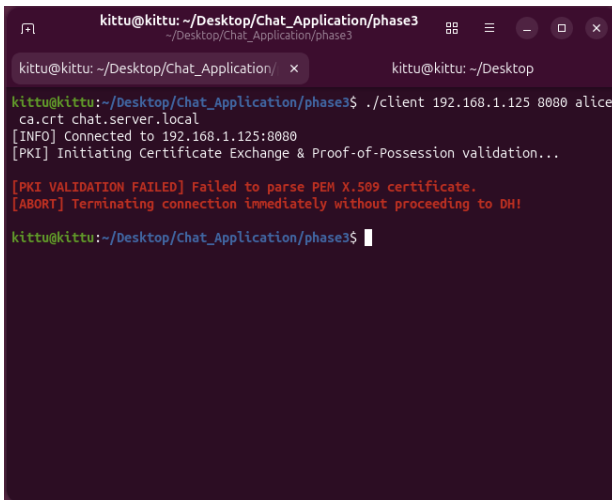
Figure 17: Phase 3: Client rejecting untrusted fake cert.

```
kittu@kittu: ~/Desktop/Chat_Application/phase3
kittu@kittu: ~/Desktop/Chat_Application/phase3$ ./mallory_proxy 8080
=====
CS6008 Phase 3 Attack Test Tool (Mallory)
Listening on port: 8080
Mode: Fake CA Certificate Attack
=====
[MALLORY] Victim client connected! Attempting Phase 3 MITM bypass...
[MALLORY] Attack Strategy: Using Mallory's self-signed fake certificate (Testing Root CA validation defense)
[MITM ATTACK DEFEATED] Client rejected Mallory's certificate/PoP and closed the socket immediately!
[MALLORY] Attack failed. No keys derived, zero plaintext intercepted.
```

Figure 18: Phase 3: Mallory logging fake cert defeat.

4.4.2 Test 2: Mallory Replaying Stolen Legitimate Certificate

Mallory presents the authentic, CA-signed `server.crt` but lacks `server.key`. When challenged to sign ($N_{client} \parallel DH_Pub$), Mallory must forge the signature using her own private key. The client extracts the authentic public key from `server.crt`, and the signature verification fails (Figures 19 and 20).



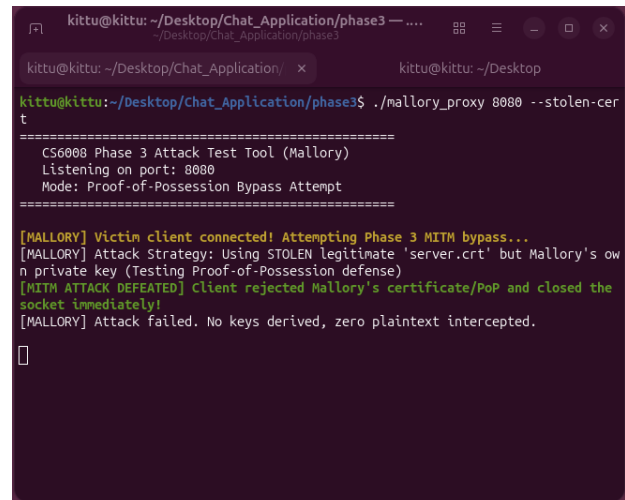
```
kittu@kittu: ~/Desktop/Chat_Application/phase3
kittu@kittu: ~/Desktop/Chat_Application/ x kittu@kittu: ~/Desktop

kittu@kittu:~/Desktop/Chat_Application/phase3$ ./client 192.168.1.125 8080 alice
ca.crt chat.server.local
[INFO] Connected to 192.168.1.125:8080
[PKI] Initiating Certificate Exchange & Proof-of-Possession validation...

[PKI VALIDATION FAILED] Failed to parse PEM X.509 certificate.
[ABORT] Terminating connection immediately without proceeding to DH!

kittu@kittu:~/Desktop/Chat_Application/phase3$
```

Figure 19: Phase 3: Client rejecting invalid PoP signature.



```
kittu@kittu: ~/Desktop/Chat_Application/phase3
kittu@kittu: ~/Desktop/Chat_Application/ x kittu@kittu: ~/Desktop

kittu@kittu:~/Desktop/Chat_Application/phase3$ ./mallory_proxy 8080 --stolen-cert
=====
CS6008 Phase 3 Attack Test Tool (Mallory)
Listening on port: 8080
Mode: Proof-of-Possession Bypass Attempt
=====

[MALLORY] Victim client connected! Attempting Phase 3 MITM bypass...
[MALLORY] Attack Strategy: Using STOLEN legitimate 'server.crt' but Mallory's own private key (Testing Proof-of-Possession defense)
[MITM ATTACK DEFEATED] Client rejected Mallory's certificate/PoP and closed the socket immediately!
[MALLORY] Attack failed. No keys derived, zero plaintext intercepted.

[]
```

Figure 20: Phase 3: Mallory logging PoP defeat.

5 Phase 4: End-to-End Encryption (E2E)

5.1 Limitations of Link-Level Security

In Phases 2 and 3, encryption terminated at the server. While protected against external eavesdroppers, the central server had full access to plaintext, representing a single point of compromise and insider threat.

5.2 E2E Handshake & Wire-Level Tagging

Phase 4 implements direct client-to-client Diffie-Hellman key agreement multiplexed across the chat server using standardized wire tags:

- `__E2E_INIT__<Alice_DH_Pub>`
- `__E2E_ACK__<Bob_DH_Pub>`
- `__E2E_MSG__<Encrypted_Payload_Hex>`

5.3 Double-Layered Encryption

Messages transmitted between peers undergo two nested layers of AES-256-GCM authenticated encryption:

1. **Inner Layer (E2E):** Plaintext is encrypted with the direct peer key K_{E2E} and wrapped in `__E2E_MSG__`.
2. **Outer Layer (Transport):** The tagged message is encrypted under the hop-by-hop link key K_{Link} established with the server.


```

kittu@kittu: ~/Desktop/Chat_Application/phase4
kittu@kittu: ~/Desktop/Chat_Application/ x    kittu@kittu: ~/Desktop

ca.crt chat.server.local
[INFO] Connected to 192.168.1.126:8080
[PKI] Server authenticated via PKI & Proof-of-Possession!
[CRYPTO] Link Key Fingerprint: 5f2a1452389848edc63171408c2f9f47343b3d0523c04efe4
12f6c64f62144c7

=====
Phase 4 Commands:
/e2e <username> - Start End-to-End Encryption with user
@username <msg> - Send message to user
/chat <username> - Switch active chat partner
/who - List online users
/quit - Disconnect and exit
=====
>
OK: Registered as alice
> @bob msg before e2e
> /e2e bob
[E2E] Initiating End-to-End handshake with @bob...
>
[E2E] Established End-to-End session with @bob!
E2E Key Fingerprint: 44e49edaf2d45e12febaa1013dbd775a550ba79ad3a948afcae
fedda87aa32
> confidential msg only to bob
>

```

Figure 21: Phase 4: Alice establishing E2E key session.

```

kittu@kittu: ~/Desktop/Chat_Application/phase4
kittu@kittu: ~/Desktop/Chat_Application/ x    kittu@kittu: ~/Desktop

a.crt chat.server.local
[INFO] Connected to 192.168.1.126:8080
[PKI] Server authenticated via PKI & Proof-of-Possession!
[CRYPTO] Link Key Fingerprint: 12b967d8f6890ed49f56851bfa5faa28fab8d0a000890e4
9e7e7516c638722

=====
Phase 4 Commands:
/e2e <username> - Start End-to-End Encryption with user
@username <msg> - Send message to user
/chat <username> - Switch active chat partner
/who - List online users
/quit - Disconnect and exit
=====
>
OK: Registered as bob
FROM @alice: msg before e2e
>
[E2E] Established End-to-End session with @alice!
E2E Key Fingerprint: 44e49edaf2d45e12febaa1013dbd775a550ba79ad3a948afcae
fedda87aa32
>
[E2E ENCRYPTED] FROM @alice: confidential msg only to bob
>

```

Figure 22: Phase 4: Bob matching E2E key & decrypting.

5.4 Verification of Server Blindness

To prove zero-knowledge server blindness, we compared server relay logs before and after triggering `/e2e bob`.

```

kittu@kittu: ~/Desktop/Chat_Application/phase4
kittu@kittu: ~/Desktop/Chat_Application/ x    kittu@kittu: ~/Desktop

kittu@kittu:~/Desktop/Chat_Application/phase4$ ./server 8080 server.crt server.
key
=====
CS6008 Phase 4 Blind Relay Server
Listening on port: 8080
=====

[INFO] New connection from 192.168.1.124:43346 (socket 4)
[PKI & CRYPTO] Client authenticated and link encrypted.
[REGISTRATION] Client on socket 4 registered as 'alice'

[INFO] New connection from 192.168.1.123:34622 (socket 5)
[PKI & CRYPTO] Client authenticated and link encrypted.
[REGISTRATION] Client on socket 5 registered as 'bob'
[RELAY] (alice -> bob): msg before e2e
[RELAY] (alice -> bob): _E2E_INIT_ 5014B516C975C1008C3B901F04ADC2D50E3EE002C525
0DE12D1947571D595209CBC5DE7D96A79397D0BF5FDA95A76D3B7CE5095AC0C362F4897920C77CAE
E7CC58B9C1E0DBAF24340D532AB8906FBE0387D5832A7635D0389581B8B2DCB85039B65D547F8432
910FB02ACB7B9EC28FF136797CE437EAD99A6700FA8BA2D98E2329E81F30EF3A79B74122EE19AF
0061704CFDE401E548F6DBE62F08D0D4382FC225B724C25E149E3208D700FDF9A5F470088F02E8C
A2566A2BE0965893E62D7358D983C1640081A285B6EECA3F0372947183CA29E8C86E4CD059965E71
A326B138B725C2088623E96FE40273C30AAE4A31D56266C1B7E0E851008C8D5018FCD966BDEB64E
1AB968E7F80771B4C99EF67E459D2509C743A995C00B42C65E0AA49FB02CCF5B2438E5F4A799562B
C70471793FC07F47E0211DD6FAE2E33E723C2686129A83494CF2D6BAA45438FE806CF5A27C02485C

```

Figure 23: Phase 4: Server relay (Pre-E2E vs `_E2E_INIT_`).

```

kittu@kittu: ~/Desktop/Chat_Application/phase4
kittu@kittu: ~/Desktop/Chat_Application/ x    kittu@kittu: ~/Desktop

1AB968E7F80771B4C99EF67E459D2509C743A995C00B42C65E0AA49FB02CCF5B2438E5F4A799562B
C70471793FC07F47E0211DD6FAE2E33E723C2686129A83494CF2D6BAA45438FE806CF5A27C02485C
458F51322E066A01A4B384BA96A08EA6113610B3E14139E2E1D562536A4853471534867EF171477D
CD66AC162C18F0E0806792523E2D7C5E53F341CE31720755561D02CA27842931B5A29C081EAE08
289B13B3D796872FF0E7017D3E0600228CD0FCB72880317B3D1C38B48BFDAS4623476AA87A48731C
11D2D5637688B3FB38C459FB0591D96CA350458855E4B10AFC343CB248D87137911ACAC10F94D56
5CCB7E3372CC58F7188
[RELAY] (bob -> alice): _E2E_ACK_ B53F2001EEF6C2A53709A448327C796A8EF5610FED8C
6B96C638C481827E08FF8E508C0DBAF4F027BD2D5D15F521FA5B1FF984C56DC27957832745F44F7E
52CA9851B626548D8275A9E977E4ED281D0F2D095DA606AC0807AB6CD9289E8C5794DA5D1158E857
C89E1152550A88E8CF2A36715E3F4C78F07D5FB9948923FE2CB051476F0FDAEFA4D04D3C99BE
6DBFDA3BB063D2C5BCBCD41F5FAFE4D11C4FFC36E25655C66006C0FAF6A47B46D8C17C23B919D39
7D780062384251E9EDADAEB61CA0EA72CEB57DD7A94A4F1260B9A9FD5FE0203570C1C6CD09F4A
E23DDA4680E9D0435516787002B88D2C729C553EAB95E24EEF834BF305E2B743440A40436F19E00
6BB804F2CAC7E472091B4015CE4FF46C4A1C899A9B776E7F181FE621FC6B4D0C1C22217EE2A0907
955AA8B98987E3432058F7576F8EC61CEB0D2002B12D886A690B2CF7B863BFBC284C70DA1C65EF3
B96C8FC71A0978BF0180D86913D472AE9F0402FBC8D238BED0EE498D3718EE54088C7A16D4E0FDA
75B1256028E2B89D0E3128D282F3AFBDC74F0D709FF19ADA8A41DEAE6AAE98EB8A0BF7CF706A56B3E
EC700F5742F0888E17B57EB5ACF833A60CD8A9DC68614E42E15664AFC4C595AC3A5A442772EE22
16608D362416EAD0ED8550310B559E8A4C737AFB14A541970345A92789C8435B3E12B12DE2977BC
21CC1CAF693302CEC036
[RELAY] (alice -> bob): _E2E_MSG_ d3eb2cf60eeFeb72d4f5ff5b1b93651db94f6df0d80ec9
7d6e78f70b48113225e307f60d5917bec08aecfdcf188ae814cbfcb4a9cc0871411

```

Figure 24: Phase 4: Server relay (`_E2E_ACK_` & `_E2E_MSG_`).

As shown in Figures 23 and 24, before E2E initiation the server logged readable chat text ("`msg before e2e`"). After `/e2e bob`, the server logs exclusively opaque, uncrackable ciphertext (`_E2E_MSG_3deb2cf60e...`), proving complete server blindness.

6 Phase 5: Forward Secrecy via Key Rotation

6.1 Threat Model & Value of Forward Secrecy

Theoretical Rationale: In Phase 4, a single static symmetric key K_{E2E} protects the entire duration of an E2E session. If an adversary passively records encrypted network traffic for days or months and later compromises the client host (recovering K_{E2E} from disk or memory), the adversary can retroactively decrypt the entire recorded session history.

Forward Secrecy ensures that compromise of a current long-term or session key does **not** compromise past session keys. Each conversation epoch is encrypted under an independent, short-lived ephemeral key that is permanently destroyed upon rotation.

6.2 Ratchet Mechanism & Memory Zeroization

Phase 5 introduces automated, periodic key rotation:

1. **Periodic Timer:** A background thread executes an ephemeral Diffie–Hellman re-exchange every 60 seconds (or immediately via `/rekey`).
2. **Memory Cleansing:** Once a new session key is derived, the previous key material and DH private exponents are immediately overwritten in RAM using `OPENSSL_cleanse()` to prevent extraction via memory dumps or cold-boot attacks.

6.3 Simultaneous Rekey Collision Avoidance

Design Justification: If Alice and Bob trigger key rotation at the exact same instant, both will transmit `__E2E_INIT__` simultaneously, risking race conditions and key desynchronization.

We resolve this through a deterministic **lexicographical tie-breaker**:

- When receiving an `__E2E_INIT__` while in the `AWAITING_ACK` state, clients compare usernames (`username_self < username_peer`).
- The client with the lexicographically higher username proceeds as the initiator; the lower-ranked client drops its pending init and yields, immediately replying with `__E2E_ACK__`.

6.4 Empirical Connection Verification

We verified three consecutive key rotations over live sessions.

Figure 25: Phase 5: Server relaying multiple periodic E2E rekey handshakes.

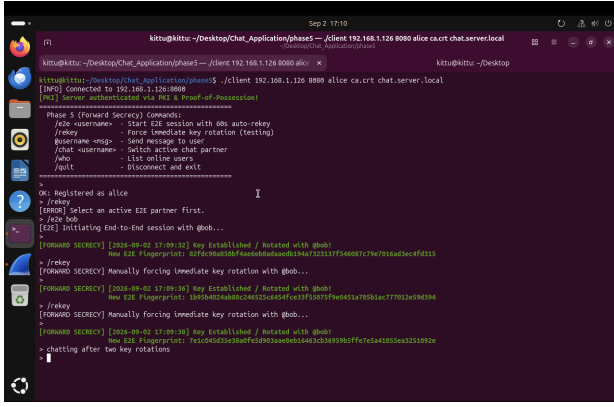


Figure 26: Phase 5: Alice log showing 2 rotations.

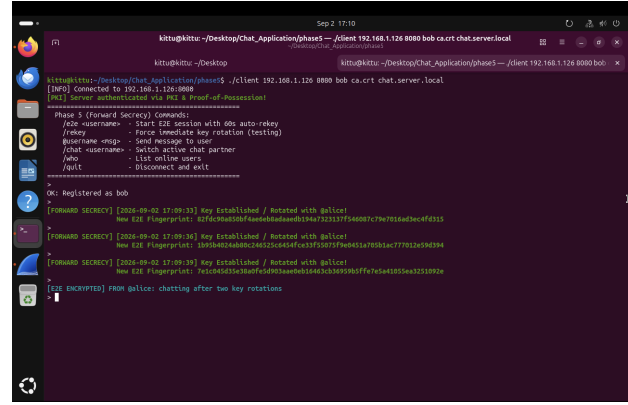


Figure 27: Phase 5: Bob log showing matching keys.

As evidenced in Figures 25, 26, and 27, both clients agree on the updated fingerprint after each rotation (82fdc9... $\rightarrow$ 1b95b4... $\rightarrow$ 7e1c04...), and messages sent after multiple rotations decrypt flawlessly.

7 Conclusion

Through the iterative development of Phases 1 to 5, we have systematically engineered a robust, modern cryptographic communication system. Starting from a vulnerable plaintext baseline, we incorporated mathematical key exchange, authenticated symmetric encryption, rigorous PKI validation, zero-knowledge blind relaying, and ratcheted forward secrecy, validating each security guarantee through empirical network captures and adversarial testing.

A Generative AI Usage & Prompt Log

In accordance with assignment policy, LLM assistance was utilized for boilerplate generation and conceptual planning. The detailed prompt sequence is documented below:

1. "How should I plan the structure of the code files so that as I finish the phases one-by-one while maintaining the modularity of the code files."
2. "Provide the OpenSSL EVP code snippet for AES-256-GCM encryption with 12-byte IV and 16-byte authentication tag retrieval."
3. "How to perform X.509 certificate verification against a Root CA store in C++ using OpenSSL low-level functions without using SSL_CTX?"
4. "Create readme.md files for each phase's folder explaining the respective folder code structure and the basic commands used for getting results."
5. "Using the Screenshots uploaded, create a template for a latex file for the report."